

FACE RECOGNITION SYSTEM

Project Documentation

A local desktop application for webcam-based face capture, LBPH model training, and real-time recognition.

Project Repository	github.com/SivasankarAddanki/face_recognition
Primary Language	Python
Computer Vision	OpenCV and OpenCV Contrib
User Interface	Tkinter
Recognition Method	Local Binary Patterns Histograms (LBPH)
Document Version	1.0 - June 2026

Prepared as technical documentation for setup, design review, demonstration, maintenance, and future development.

Table Of Contents

Table Of Contents	2
1. Executive Summary	4
2. Objectives And Scope	4
2.1 Objectives	4
2.2 Scope	4
3. Technology Stack	4
4. System Requirements	4
5. Architecture	6
5.1 Logical Architecture	6
5.2 Recognition Data Flow	6
5.3 Training Data Flow	6
6. Project Structure	6
7. Module Design	6
8. Face Detection And Preprocessing	7
8.1 Haar Cascade Detection	7
8.2 Standardization	7
8.3 Lighting Normalization	7
9. LBPH Recognition	8
10. Installation And Execution	8
10.1 Clone And Configure	8
10.2 Run	8
11. Operating Procedure	8
11.1 Add Or Replace A Person	8
11.2 Recognize A Person	8
12. Data And Model Management	8
12.1 Current Repository State	9
13. Testing Strategy	10
13.1 Functional Tests	10
13.2 Evaluation Method	10
14. Error Handling And Troubleshooting	10
15. Security, Privacy, And Ethics	10
16. Limitations	10

17. Future Enhancements	11
18. Conclusion	11
Appendix A: Key Configuration	11
Appendix B: Repository	11

1. Executive Summary

The Face Recognition System is a small offline desktop application that captures facial images from a webcam, trains an LBPH recognition model, and identifies trained people in live video. Haar Cascade classification detects frontal faces. Tkinter provides a minimal interface with separate capture/training and recognition actions.

The system is designed for learning and small local demonstrations. It does not depend on cloud services. Face images, label mappings, and the trained model are stored inside the project directory.

Responsible-use note: This project processes biometric data. It should only be used with informed permission and should not be used for security-critical or high-impact decisions.

2. Objectives And Scope

2.1 Objectives

- Detect a frontal face using a webcam.
- Capture and organize 100 face samples per person.
- Normalize images to improve consistency across lighting conditions.
- Train and save an LBPH face-recognition model.
- Recognize known faces and label weak matches as Unknown.
- Provide a simple interface suitable for demonstration and learning.

2.2 Scope

The project targets a small number of people in controlled indoor conditions. It supports one local webcam and performs processing on the CPU. It does not provide user authentication, liveness detection, remote storage, access control, or large-scale identity search.

3. Technology Stack

Component	Technology	Role
Language	Python	Application logic and orchestration
Computer vision	OpenCV	Camera, detection, image processing
Recognition	OpenCV Contrib LBPH	Training and prediction
Numeric data	NumPy	Integer label arrays for model training
Interface	Tkinter	Desktop controls and status messages
Detection model	Haar Cascade XML	Frontal-face localization
Storage	JPEG, YAML, JSON	Samples, model, and name mapping

4. System Requirements

Type	Requirement
Operating system	Windows 10 or Windows 11
Python	Python 3.10 or newer

Type	Requirement
Camera	Integrated or USB webcam
Packages	numpy and opencv-contrib-python
Memory	At least 4 GB recommended
Storage	Approximately 100 MB plus future face samples

5. Architecture

5.1 Logical Architecture

User / Webcam
Tkinter Application
OpenCV Camera And Haar Face Detector
Preprocessing: Crop, Resize, Histogram Equalization
LBPH Training Or Prediction
Dataset, trainer.yml, and labels.json

5.2 Recognition Data Flow

Webcam frame -> grayscale conversion -> Haar detection -> face crop -> 200 x 200 resize -> histogram equalization -> LBPH prediction -> label lookup -> known name or Unknown

5.3 Training Data Flow

Person name -> dataset folder -> 100 face samples -> preprocessing -> numeric labels -> LBPH training -> trainer.yml + labels.json

6. Project Structure

```
face_recognition/
|-- dataset/<person_name>/*.jpg
|-- docs/
|-- model/trainer.yml
|-- model/labels.json
|-- haarcascade_frontalface_default.xml
|-- train.py
|-- requirements.txt
|-- run_face_recognition.bat
`-- README.md
```

7. Module Design

Function / Class	Responsibility
preprocess_face	Resize face to 200 x 200 and equalize contrast
expand_person_dataset	Create mild variations when a dataset has fewer than 100 images
create_recognizer	Validate cv2.face and create an LBPH recognizer
open_camera	Open camera with Windows fallback handling
load_detector	Load and validate the Haar Cascade
safe_folder_name	Convert a supplied name into a safe folder name
capture_face	Detect, crop, display, and save face samples
train_model	Read datasets, train LBPH, and save model artifacts
recognize_faces	Run real-time prediction and display results

Function / Class	Responsibility
FaceRecognitionApp	Build the Tkinter interface and button actions

8. Face Detection And Preprocessing

8.1 Haar Cascade Detection

The Haar Cascade uses learned rectangular intensity features to scan a grayscale image for frontal faces. The application uses a minimum face size of 100 x 100 pixels and selects the largest detected face during sample capture.

8.2 Standardization

Each cropped face is resized to 200 x 200 pixels. Consistent dimensions make local texture regions comparable during LBPH training and prediction.

8.3 Lighting Normalization

Histogram equalization redistributes grayscale intensity values to improve contrast. This reduces some variation between brighter and darker captures, although it cannot solve every lighting problem.

9. LBPH Recognition

Local Binary Patterns Histograms compare each pixel with neighboring pixels to encode local texture. Histograms summarize these patterns across regions of the face. During prediction, OpenCV returns the best numeric label and a distance value.

Distance interpretation: Lower values indicate a closer match. This project uses a threshold of 65.0. A predicted distance above the threshold is displayed as Unknown.

LBPH is fast and practical for a classroom-scale offline system. Its accuracy can decrease with large pose changes, occlusion, aging, poor image quality, or major differences between training and runtime conditions.

10. Installation And Execution

10.1 Clone And Configure

```
git clone https://github.com/SivasankarAddanki/face_recognition.git
cd face_recognition
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
pip install -r requirements.txt
```

10.2 Run

```
python train.py

Alternative without activation:
.\.venv\Scripts\python.exe train.py
```

Windows users can also double-click **run_face_recognition.bat** after the virtual environment has been created.

11. Operating Procedure

11.1 Add Or Replace A Person

- Start the application.
- Enter the person's name.
- Click Capture Face and Train.
- Keep one face visible and slowly vary head angle and expression.
- Wait until 100 samples are captured and training completes.
- Confirm that trainer.yml and labels.json have updated timestamps.

11.2 Recognize A Person

- Click Recognize Face.
- Look toward the webcam under similar lighting.
- Read the name and distance displayed over the detected face.
- Press Q while the camera window is focused to stop.

12. Data And Model Management

Each non-empty folder under dataset represents one identity. Folder names become readable labels. Numeric label assignments are saved in labels.json, while learned LBPH histograms are saved in trainer.yml.

Retraining replaces the previous model. Back up a known-good model before experimenting with poor-quality data. Never assume that the YAML model can recreate deleted source images.

12.1 Current Repository State

The supplied saved model lists praveen, shiva, and vinay. At document generation time, the repository contained 100 source images for Praveen. Shiva and Vinay remain encoded in the existing trained model, but their original source images were no longer available. Retraining from the current repository will use only non-empty dataset folders.

13. Testing Strategy

13.1 Functional Tests

Test	Expected Result
Launch application	Tkinter window opens without an exception
Open camera	Live video appears and frames are returned
Detect frontal face	Rectangle appears around the face
Capture samples	100 JPEG files are saved for a new person
Train model	trainer.yml and labels.json are created or replaced
Known face	Correct name appears when distance is within threshold
Unknown face	Unknown appears when the threshold is exceeded
Quit camera	Q closes the active OpenCV window

13.2 Evaluation Method

Model quality should be measured with face images excluded from training. Split original captures into training and test groups, then report per-person precision, recall, a confusion matrix, and unknown-face false acceptance. Testing the model on its own training images does not provide a reliable accuracy estimate.

14. Error Handling And Troubleshooting

Problem	Likely Cause	Resolution
Camera does not open	Camera busy or permission denied	Close other apps and allow Windows camera access
cv2.face missing	Wrong OpenCV package	Install opencv-contrib-python
No face detected	Pose, light, or distance	Face camera directly and improve lighting
Wrong identity	Weak training variation or loose threshold	Recapture data and evaluate threshold
Only one label after retraining	Other dataset folders are empty	Restore or recapture missing source images
PowerShell activation blocked	Execution policy	Use process-scoped Bypass or call venv Python directly

15. Security, Privacy, And Ethics

- Obtain informed consent before capturing a person's face.
- Treat face images and trained models as sensitive biometric data.
- Do not publish datasets without permission from every represented person.
- Restrict repository and filesystem access when real biometric data is present.
- Delete samples and models when the stated purpose is complete.
- Do not use this learning project for policing, surveillance, access control, or consequential decisions.

16. Limitations

- The detector is optimized for frontal faces.
- The system has no liveness detection and may accept a photograph.
- Consecutive capture frames may contain limited variation.
- LBPH is less robust than modern embedding models.
- The GUI performs camera work on the main thread.
- The threshold is fixed and has not been calibrated for every environment.
- The application is intended for small local datasets.

17. Future Enhancements

- Add blur, exposure, and face-angle quality checks.
- Capture samples at controlled time intervals.
- Add a dedicated train/test split and evaluation report.
- Use a DNN-based face detector.
- Use modern face embeddings for stronger identity comparison.
- Add liveness and anti-spoofing checks.
- Add dataset management and model backup controls.
- Move camera processing to a background worker.
- Encrypt or exclude biometric artifacts from public repositories.

18. Conclusion

The project demonstrates the complete classical computer-vision pipeline: acquisition, detection, preprocessing, supervised training, model persistence, and real-time prediction. Its compact design makes it useful for learning OpenCV concepts and desktop integration. For production use, stronger recognition, liveness, evaluation, privacy, and security controls would be required.

Appendix A: Key Configuration

Setting	Value
Samples per person	100
Face dimensions	200 x 200 pixels
Recognition threshold	65.0 LBPH distance
Camera index	0
Model path	model/trainer.yml
Label path	model/labels.json
Detector path	haarcascade_frontalface_default.xml

Appendix B: Repository

Source repository:

https://github.com/SivasankarAddanki/face_recognition